Data and Algorithm Analysis

Chapter 34 — NP-Completeness

Thang Hoang

Department of Computer Science, Virginia Tech

Copyright © Lenwood S. Heath, 2017

Outline

Introduction

34.1 Polynomial Time

34.2 Polynomial-Time Verification

34.3 NP-Completeness

34.4 Proofs

34.5 Problems

Table of Contents

Introduction

34.1 Polynomial Time

34.2 Polynomial-Time Verification

34.3 NP-Completeness

34.4 Proofs

34.5 Problems

Complexity Theory

Complexity theory is the mathematical study of classes of problems defined by their time or space complexity.

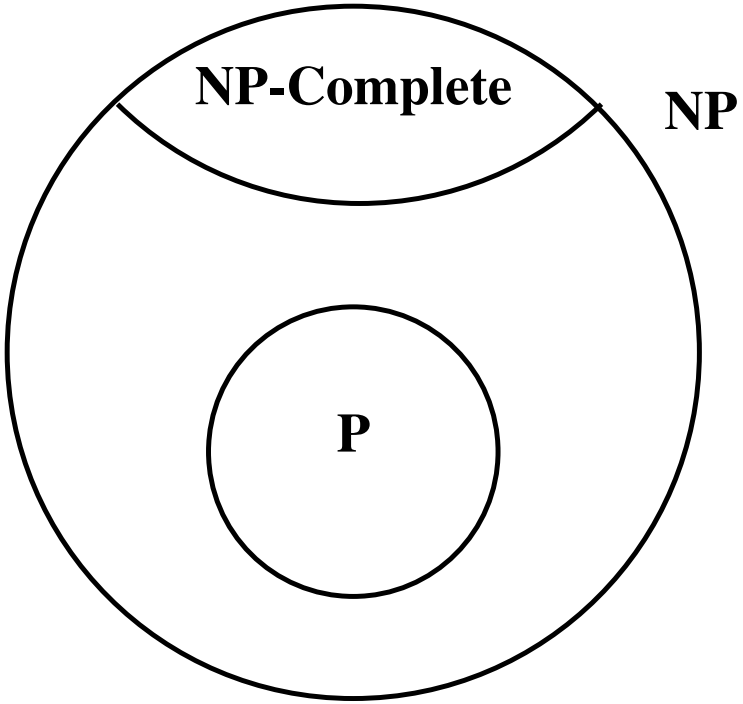
Many classes of problems are studied, e.g.,

- Graph problems
- Scheduling problems
- Ordering problems (sorting)
- Searching problems
- Boolean logic — both formulas and circuits
- Combinatorial, such as 0-1-KNAPSACK

Complexity Classes

We will study

1. P — problems solvable in polynomial time
2. NP — problems verifiable in polynomial time
3. NP-complete — hardest problems in NP



Our Approach — Complexity Theory

- Decision problems versus optimization problems
  - A **decision problem** is one where there are only two possible solutions: YES or NO; TRUE or FALSE; 1 or 0.
- Formal language framework
- Encoding instances of problems
- Verification
- Reduction of one problem to another in polynomial time

Some Graph Problems — Optimization

SHORTEST-PATH

INSTANCE: Directed graph $G = (V, E)$; $u, v \in V$.

SOLUTION: A simple path from u to v having the fewest edges possible.

LONGEST-PATH

INSTANCE: Directed graph $G = (V, E)$; $u, v \in V$.

SOLUTION: A simple path from u to v having the most edges possible.

Some Graph Problems — Decision

EULER-TOUR

INSTANCE: Directed graph $G = (V, E)$.

QUESTION: Does G contain an Eulerian tour, a cycle containing every edge exactly once?

HAMILTONIAN-CYCLE

INSTANCE: Directed graph $G = (V, E)$.

QUESTION: Does G contain a Hamiltonian cycle, a cycle containing every vertex exactly once?

Converting Optimization to Decision

DECISION-SHORTEST-PATH

INSTANCE: Directed graph $G = (V, E)$; $u, v \in V$; integer $k \geq 0$.

QUESTION: Is there a simple path from u to v having $\leq k$ edges?

DECISION-LONGEST-PATH

INSTANCE: Directed graph $G = (V, E)$; $u, v \in V$; integer $k \geq 0$.

QUESTION: Is there a simple path from u to v having $\geq k$ edges?

Convert an optimization problem to a decision problem by adding a parameter to bound the objective function, in this case k to bound the length of a path.

Solving an Optimization Problem Using a Decision Problem

DECISION-LONGEST-PATH

INSTANCE: Directed graph $G = (V, E)$; $u, v \in V$; integer $k \geq 0$.

QUESTION: Is there a simple path from u to v having $\geq k$ edges?

LONGEST-PATH

INSTANCE: Directed graph $G = (V, E)$; $u, v \in V$.

SOLUTION: A simple path from u to v having the most edges possible.

Given an algorithm for DECISION-LONGEST-PATH as a subroutine, solve LONGEST-PATH.

Natural Decision Problems — Boolean Logic

Boolean Connectives: AND ($\wedge$); OR ($\vee$); NOT ($\neg$)

A boolean formula $\phi(x_1, x_2, \ldots, x_n)$ is in **conjunctive normal form (CNF)** if it is the AND of **clauses,** each of which is an OR of **literals,** each of which is a variable $x_1, x_2, \ldots, x_n$ or its negation.

Example

$$\phi_1(x_1, x_2, x_3, x_4) \quad = \quad (x_1 \vee \neg x_2 \vee x_3 \vee \neg x_4) \wedge (x_2 \vee \neg x_3)$$

The boolean formula $\phi(x_1, x_2, \ldots, x_n)$ is **satisfiable** if there is an assignment of 0 and 1 values to $x_1, x_2, \ldots, x_n$ that makes ϕ TRUE (equal to 1).

Formulas in k-CNF

A boolean formula $\phi(x_1, x_2, \ldots, x_n)$ in conjunctive normal form (CNF) is in k**-CNF** if each clause has exactly k literals.

Example

$$\phi_2(x_1, x_2, x_3, x_4) \quad = \quad (x_1 \vee \neg x_2) \wedge (x_1 \vee x_3) \wedge (\neg x_3 \vee x_4)$$

Is ϕ_2 satisfiable? What is the evidence?

Satisfiability Problems

2-CNF-SAT
INSTANCE: Boolean formula $\phi(x_1, x_2, \ldots, x_n)$ in 2-CNF.
QUESTION: Is ϕ satisfiable?

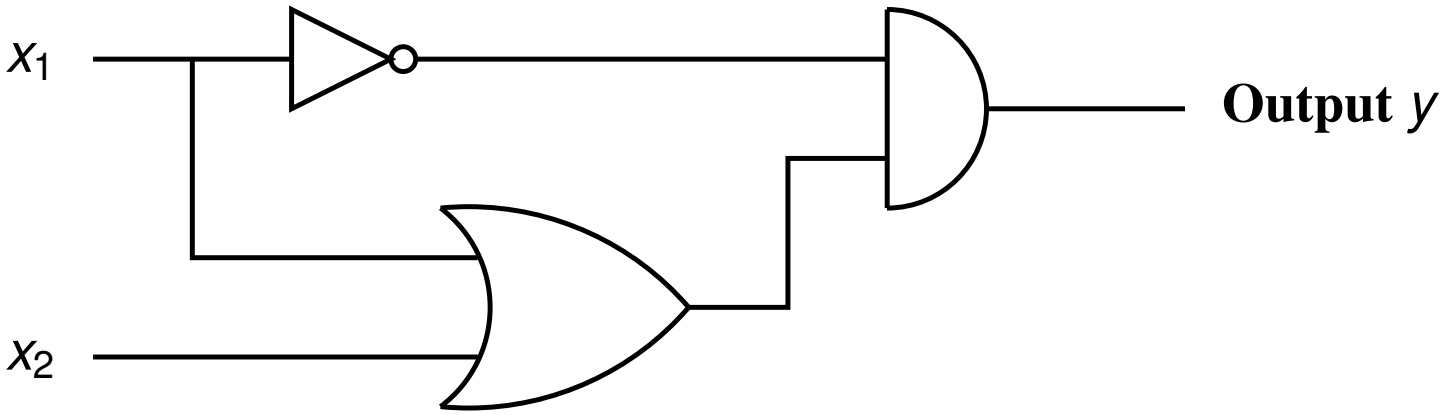
3-CNF-SAT
INSTANCE: Boolean formula $\phi(x_1, x_2, \ldots, x_n)$ in 3-CNF.
QUESTION: Is ϕ satisfiable?

SAT
INSTANCE: Boolean formula $\phi(x_1, x_2, \ldots, x_n)$ in CNF.
QUESTION: Is ϕ satisfiable?

It will turn out that 2-CNF-SAT is in P, while 3-CNF-SAT and SAT are NP-complete.

Boolean Combinatorial Circuits

Using logic gates AND, OR, and NOT, a boolean combinatorial circuit has inputs $x_1, x_2, \ldots, x_n$, one output y, and no cycles (the graph is a DAG).



The boolean combinatorial circuit $C(x_1, x_2, \ldots, x_n)$ is **satisfiable** if there is an assignment of 0 and 1 values to $x_1, x_2, \ldots, x_n$ that makes C TRUE (equal to 1).

Circuit Satisfiability Problem

CIRCUIT-SAT
INSTANCE: Boolean combinatorial circuit $C(x_1, x_2, \ldots, x_n)$.
QUESTION: Is C satisfiable?

It will turn out that CIRCUIT-SAT is NP-complete. In fact, it will be our first NP-complete problem.

Table of Contents

Introduction

34.1 Polynomial Time

34.2 Polynomial-Time Verification

34.3 NP-Completeness

34.4 Proofs

34.5 Problems

Formal Language Framework

An **alphabet** Σ is a finite set of symbols or characters. For example, $\Sigma = \{0, 1\}$, set of letters, and set of ASCII characters.

A **string** $\alpha = \sigma_1\sigma_2 \cdots \sigma_k$ over Σ is a finite sequence of symbols from Σ. Its **length** is k. For example, 1011, a string of length 4; ϵ, the string of length 0; and 0, a string of length 1.

Concatenation is an associative binary operation on strings.

$$(\sigma_1\sigma_2 \cdots \sigma_s) \cdot (\tau_1\tau_2 \cdots \tau_t) \quad = \quad \sigma_1\sigma_2 \cdots \sigma_s\tau_1\tau_2 \cdots \tau_t$$

is a string of length $s + t$.

Languages

A **language** L over Σ is a set of strings over Σ.

Example

$\emptyset$, the empty language
$\{1001, 010, \epsilon\}$, a finite language
$\{10^k \mid k \geq 0\}$, an infinite language
Σ^*, the language of all strings over Σ

Operations on Languages

The **complement** of language L is $\overline{L} = \Sigma^* \setminus L$.

The **concatenation** of languages L_1 and L_2 is

$$L_1 L_2 \quad = \quad \{x_1 x_2 \mid x_1 \in L_1 \text{ and } x_2 \in L_2\}.$$

Using concatenation of a language L with itself, we can define powers L^i, for $i \geq 0$, where $L^0 = \{\epsilon\}$.

The **Kleene closure** or **Kleene star** of language L is

$$L^* \quad = \quad \bigcup_{i=0}^{\infty} L^i \quad = \quad \{\epsilon\} \cup L \cup L^2 \cup L^3 \cup \cdots .$$

Encoding Instances as Strings

We want to represent decision problems as languages. To do so, we have to encode the instances of a decision problem as strings; we will use the binary alphabet $\Sigma = \{0, 1\}$. We will use ρ for the encoding function.

How can we encode an instance $\phi(x_1, x_2, \ldots, x_n)$ of SAT as a binary string?

Example

CNF formula ϕ where $n = 5$:

$$(x_1 \vee \neg x_2 \vee x_4) \wedge (x_2 \vee x_3) \wedge (x_1 \vee \neg x_4 \vee x_5)$$

Thoughts on encoding?

Encoding Variables

Since there are n variables, we need $k = \lceil \lg n \rceil$ bits to encode the variables, that is, each variable is encoded as a string of length k.

$$(x_1 \vee \neg x_2 \vee x_4) \wedge (x_2 \vee x_3) \wedge (x_1 \vee \neg x_4 \vee x_5)$$

Here $n = 5$, so $k = 3$. One easy way to encode the variables is as follows:

Variable	Encoding
x_1	000
x_2	001
x_3	010
x_4	011
x_5	100

Encoding Literals

We need 1 additional bit to encode not negated or negated, so we need $k + 1$ bits to encode a literal.

$$(x_1 \vee \neg x_2 \vee x_4) \wedge (x_2 \vee x_3) \wedge (x_1 \vee \neg x_4 \vee x_5)$$

Here is a reasonable encoding:

Not Negated	Encoding
x_1	0000
x_2	0001
x_3	0010
x_4	0011
x_5	0100

Negated	Encoding
$\neg x_1$	1000
$\neg x_2$	1001
$\neg x_3$	1010
$\neg x_4$	1011
$\neg x_5$	1100

Encoding AND and OR

We need one more bit to encode $\wedge$ (AND) versus $\vee$ (OR), say, $\wedge$ is 0, and $\vee$ is 1.

$$(x_1 \vee \neg x_2 \vee x_4) \wedge (x_2 \vee x_3) \wedge (x_1 \vee \neg x_4 \vee x_5)$$

Encoding the boolean formula requires concatenating the components identified so far:

$$0000 \cdot 1 \cdot 1001 \cdot 1 \cdot 0011 \cdot 0 \cdot 0001 \cdot 1 \cdot 0010 \cdot 0 \cdot 0000 \cdot 1 \cdot 1011 \cdot 1 \cdot 0100.$$

This yields the string

$$\alpha = 0000110011001100001100100000011011101 00.$$

Putting It All Together — Encoding

To unambiguously decode α, we need to know k. Here, we can use unary to encode k. For example, we can encode our instance of SAT as

$$\rho(\phi) \quad = \quad 0^k 1 \alpha.$$

$$(x_1 \vee \neg x_2 \vee x_4) \wedge (x_2 \vee x_3) \wedge (x_1 \vee \neg x_4 \vee x_5)$$

In summary, we encode this instance as

$$\rho(\phi) \quad = \quad 000100001100110011000011001000000110111 0100.$$

The length of the encoding is proportional to the length of the formula times $\lg n$.

Decision Problems as Languages

For any decision problem and appropriate encoding ρ, we define a language that consists of the encodings of YES-instances.

$$\text{DECISION-SHORTEST-PATH} = \{\rho(G, u, v, k) \mid G \text{ contains a path from } u \text{ to } v \text{ of length} \leq k\}$$

$$\text{2-CNF-SAT} = \{\rho(\phi) \mid \phi \text{ is a satisfiable 2-CNF formula}\}$$

$$\text{SAT} = \{\rho(\phi) \mid \phi \text{ is a satisfiable CNF formula}\}$$

Polynomial-Time Solvable

A language $L \subseteq \{0,1\}^*$ is **accepted** by an algorithm A if, for each $x \in \{0,1\}^*$, $A(x) = \textsc{True}$ if and only if $x \in L$.

A language L is **solvable in polynomial time** if there exists an integer $k \geq 0$ and an algorithm A that solves (accepts) L in time complexity $O(n^k)$, where n is the size a the string in $\{0,1\}^*$ given as input to A.

The **class of polynomial-time solvable languages** is

$$\mathrm{P} \quad = \quad \{L \subseteq \{0,1\}^* \mid L \text{ is solvable in polynomial time}\}.$$

This is considered the class of computationally tractable languages.

Examples — Polynomial-Time Solvable

Because of breadth-first search, we have

$$\textsc{Decision-Shortest-Path} \in \mathrm{P}.$$

Also, we have

$$\text{2-CNF-}\textsc{Sat} \in \mathrm{P}.$$

We believe that

$$\textsc{Sat} \notin \mathrm{P}$$

and will give evidence by showing that SAT is NP-complete.

Table of Contents

Introduction

34.1 Polynomial Time

34.2 Polynomial-Time Verification

34.3 NP-Completeness

34.4 Proofs

34.5 Problems

Motivation — SAT

SAT
INSTANCE: Boolean formula $\phi(x_1, x_2, \ldots, x_n)$ in CNF.
QUESTION: Is ϕ satisfiable?

Observations

- SAT is not known to be in P.
- If a particular truth assignment to $x_1, x_2, \ldots, x_n$ makes $\phi(x_1, x_2, \ldots, x_n)$ TRUE, it is easy to verify this in polynomial time.
- Such a truth assignment provides proof that $\phi \in$ SAT.

Computational Verification

Let $x, y \in \{0,1\}^*$. Let $A(x, y)$ be an algorithm that always returns 0 or 1. Call y a **certificate** for x.

The language **verified** by A is

$$L_A = \{x \in \{0,1\}^* \mid \text{there exists a certificate } y \in \{0,1\}^* \text{ such that } A(x,y) = 1\}.$$

Example

An algorithm $A(\phi(x_1, x_2, \ldots, x_n), y)$, where $y \in \{0,1\}^n$, that evaluates $\phi(y_1, y_2, \ldots, y_n)$ verifies SAT, that is, $L_A = \text{SAT}$.

Polynomial-Time Verification

Algorithm A **verifies** L_A **in polynomial time** if $A(x, y)$ has time complexity polynomial in $|x| + |y|$ and if, for every $x_1 \in L_A$, there exists a certificate $y_1 \in \{0, 1\}^*$ such that $A(x_1, y_1) = 1$ and $|y_1| \in O(|x_1|^k)$, for some fixed $k \geq 0$.

In other words, the certificate y must be of length polynomial in the length of x.

Nondeterministic Polynomial Time

The class **nondeterministic polynomial time** is

$$\mathrm{NP} \quad = \quad \{L_A \mid A \text{ is a polynomial time verifier of } L_A\}.$$

We have already demonstrated an A that shows that

$$\textsc{Sat} \in \mathrm{NP}.$$

There are many other examples:

$$\begin{aligned} \textsc{Decision-Longest-Path} &\in \mathrm{NP} \\ \textsc{3-CNF-Sat} &\in \mathrm{NP} \\ &\vdots \end{aligned}$$

These problems are not known to be in P.

Table of Contents

Introduction

34.1 Polynomial Time

34.2 Polynomial-Time Verification

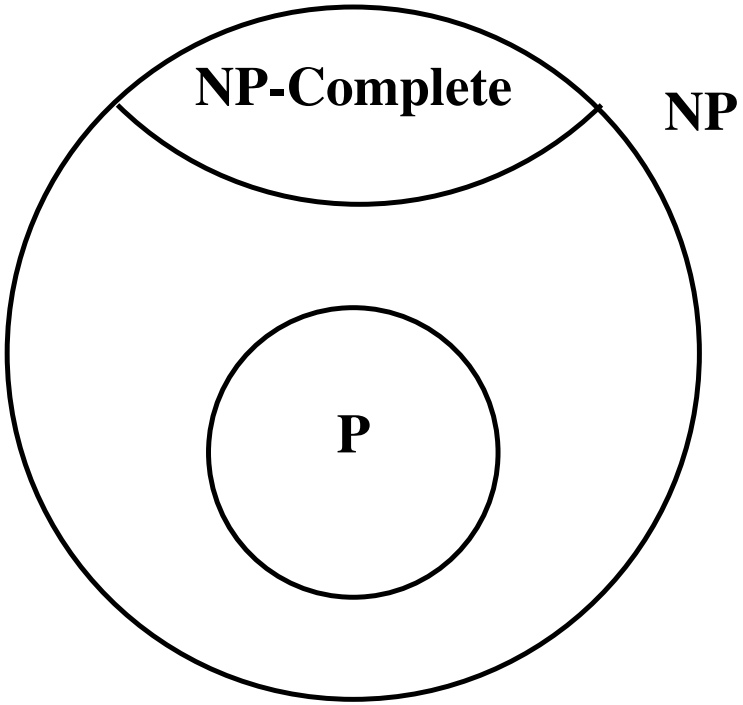
34.3 NP-Completeness

34.4 Proofs

34.5 Problems

The NP-Completeness Picture

If $\mathrm{P} \neq \mathrm{NP}$, then this is the complexity class picture:



Of course, if $\mathrm{P} = \mathrm{NP}$, then every polynomial time verifiable language is accepted in polynomial time; this does not seem to be the case.

Overview — NP-Completeness

- Through instance encoding, we represent decision problems as languages.
- Through polynomial-time reduction, we relate the time complexity of languages.

Example

$$\text{3-CNF-SAT} = \{\rho(\phi) \mid \phi \text{ is a satisfiable 3-CNF formula}\}$$

$$\text{CIRCUIT-SAT} = \{\rho(C) \mid C \text{ is a satisfiable boolean combinatorial circuit}\}$$

Reduction

Let $L_1, L_2 \subseteq \{0,1\}^*$. A function $f : \{0,1\}^* \to \{0,1\}^*$ is a **reduction** if f is computable by an algorithm F and if $f(\alpha) \in L_2$ if and only if $\alpha \in L_1$.

Suppose A_2 is an algorithm that accepts L_2. Then, we have this algorithmic diagram:

$$L_1 \xrightarrow{f} L_2 \xrightarrow{A_2} \{0,1\}$$

In particular, we can define a composition algorithm $A_1(\alpha) = A_2(f(\alpha))$ that accepts L_1.
We write $L_1 \leq L_2$, read "L_1 reduces to L_2".

Polynomial-Time Reduction

If the algorithm F of the reduction is a polynomial-time algorithm, then f is a **polynomial-time reduction of L_1 to L_2.** We write $L_1 \leq_{\mathrm{P}} L_2$, read "$L_1$ reduces in polynomial time to L_2".

In the composition $A_1(\alpha) = A_2(f(\alpha))$, if both A_2 and f are polynomial time, then A_1 is a polynomial-time algorithm.

Lemma

If $L_2 \in \mathrm{P}$ and $L_1 \leq_{\mathrm{P}} L_2$, then $L_1 \in \mathrm{P}$.

NP-Completeness

A language $L \subseteq \{0,1\}^*$ is **NP-complete** if $L \in \mathrm{NP}$ and, for every $L' \in \mathrm{NP}$, we have $L' \leq_{\mathrm{P}} L$.

Theorem (Cook-Levin Theorem)

SAT *is NP-complete.*

Theorem (34.7)

CIRCUIT-SAT *is NP-complete.*

Significance — NP-Completeness

Theorem (34.4)

If any NP-complete problem is in P, then $\mathrm{P} = \mathrm{NP}$.

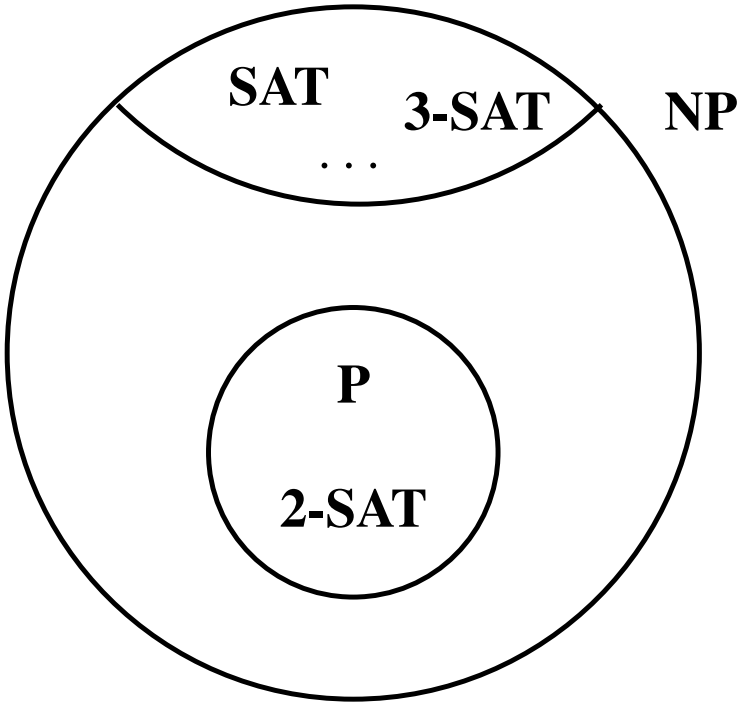


Table of Contents

Introduction

34.1 Polynomial Time

34.2 Polynomial-Time Verification

34.3 NP-Completeness

34.4 Proofs

34.5 Problems

Lemma 34.8

Lemma

If L' is NP-complete, $L \in \mathrm{NP}$, and $L' \leq_{\mathrm{P}} L$, then L is NP-complete.

The lemma provides a recipe for proving additional problems NP-complete, given the Cook-Levin Theorem.

NP-Completeness Proof Paradigm

To show a language L is NP-complete.

1. Show $L \in \mathrm{NP}$. Identify the certificate required to verify a string $x \in L$. Certificate must have length polynomial in $|x|$.
2. Select a known NP-complete language L'.
3. To show $L' \leq_{\mathrm{P}} L$, build a function $f : \{0,1\}^* \to \{0,1\}^*$ to be the reduction.
4. Prove that $x \in L'$ if and only if $f(x) \in L$. This proves $L' \leq L$.
5. Prove that f can be computed in polynomial time. This proves $L' \leq_{\mathrm{P}} L$.
6. Conclude that L is NP-complete.

Theorem 34.10.

Theorem

3-CNF-SAT *is NP-complete.*

Proof sketch.

1. Show 3-CNF-SAT $\in$ NP.
2. Select the known NP-complete language SAT.
3. Build a reduction f that takes an instance of SAT and returns an instance of 3-CNF-SAT.
4. Prove $x \in$ SAT if and only if $f(x) \in$ 3-CNF-SAT.
5. Prove that f can be computed in polynomial time.
6. Conclude that 3-CNF-SAT is NP-complete.

Proof of Theorem 34.10.

1. Show 3-CNF-SAT $\in$ NP.

$$\text{3-CNF-SAT} = \{\rho(\phi) \mid \phi \text{ is a satisfiable 3-CNF formula}\}$$

Need a certificate whose length is polynomial in $|\rho(\phi)|$ and that allows verification in polynomial time.

What is the right certificate?

Proof of Theorem 34.10.

2. Select the known NP-complete language SAT.
Here it is:

$$\text{SAT} = \{\rho(\phi) \mid \phi \text{ is a satisfiable CNF formula}\}$$

By the Cook-Levin Theorem, SAT is NP-complete.

Proof of Theorem 34.10.

3. Build a reduction f that takes an instance of SAT and returns an instance of 3-CNF-SAT.
Let $\phi(x_1, x_2, \ldots, x_n)$ be an instance of SAT, a CNF formula with m clauses. We may express ϕ this way:

$$\phi(x_1, x_2, \ldots, x_n) = \bigwedge_{j=1}^{m} (y_{j1} \vee y_{j2} \vee \cdots \vee y_{jk_j}),$$

where each y_{ji} is a literal.

Proof of Theorem 34.10.

Example

$$\begin{aligned}\phi(x_1, x_2, x_3, x_4, x_5) &= (\neg x_1 \vee x_2 \vee \neg x_3 \vee x_4 \vee x_5) \wedge \\ &\quad (x_1 \vee \neg x_2) \wedge (x_2 \vee \neg x_4 \vee \neg x_5)\end{aligned}$$

Here, $n = 5$ and $m = 3$.

Proof of Theorem 34.10.

We need a 3-CNF $\phi'(x_1, x_2, \ldots, x_n, \ldots)$ that is satisfiable if and only if ϕ is satisfiable. Convert ϕ to ϕ' clause by clause.

For simplicity, let $(a_1 \vee a_2 \vee \cdots \vee a_\ell)$ be a clause of ϕ, where each a_i is a literal over $x_1, x_2, \ldots, x_n$. Depending on ℓ, there are four cases.

$\ell = 3$. There is nothing to do. The converted clause is the same as the original clause.

$\ell = 2$. The original clause converts to two clauses $(a_1 \vee a_2 \vee w_1) \wedge (a_1 \vee a_2 \vee \neg w_1)$, where w_1 is a new variable.

Proof of Theorem 34.10.

$\ell = 1$. The original clause converts to four clauses
$(a_1 \vee u_1 \vee v_1) \wedge (a_1 \vee u_1 \vee \neg v_1) \wedge (a_1 \vee \neg u_1 \vee v_1) \wedge (a_1 \vee \neg u_1 \vee \neg v_1)$,
where u_1 and v_1 are new variables.

$\ell > 3$. The original clause converts to $\ell - 2$ clauses

$$(a_1 \vee a_2 \vee z_1) \wedge (\neg z_1 \vee a_3 \vee z_2) \wedge (\neg z_2 \vee a_4 \vee z_3) \wedge \cdots$$
$$(\neg z_{\ell-3} \vee a_{\ell-1} \vee a_\ell),$$

where $z_1, z_2, \ldots, z_{\ell-3}$ are new variables.

Proof of Theorem 34.10.

Example

$$\phi(x_1, x_2, x_3, x_4, x_5) = (\neg x_1 \vee x_2 \vee \neg x_3 \vee x_4 \vee x_5) \wedge (x_1 \vee \neg x_2) \wedge (x_2 \vee \neg x_4 \vee \neg x_5)$$

converts to

$$\phi'(x_1, x_2, x_3, x_4, x_5, w_1, z_1, z_2) = (\neg x_1 \vee x_2 \vee z_1) \wedge (\neg z_1 \vee \neg x_3 \vee z_2) \wedge (\neg z_2 \vee x_4 \vee x_5) \wedge (x_1 \vee \neg x_2 \vee w_1) \wedge (x_1 \vee \neg x_2 \vee \neg w_1) \wedge (x_2 \vee \neg x_4 \vee \neg x_5).$$

Proof of Theorem 34.10.

4. Prove $x \in$ SAT if and only if $f(x) \in$ 3-CNF-SAT.

The key idea is to take the four cases for f and argue the desired "if and only if" result for each.

We can do this verbally at the same time we construct ϕ'. Let's go back and do that.

Proof of Theorem 34.10.

5. Prove that f can be computed in polynomial time.

Since f is a simple transformation and the length of $\rho(\phi')$ is polynomial in the length of $\rho(\phi)$, it is clear that f can be computed in polynomial time.

Proof of Theorem 34.10.

6. Conclude that 3-CNF-SAT is NP-complete.

Since 3-CNF-SAT $\in \mathrm{NP}$, SAT is NP-complete, and SAT $\leq_{\mathrm{P}}$ 3-CNF-SAT, we conclude that 3-CNF-SAT is NP-complete, by Lemma 34.8.

This completes the proof of Theorem 34.10.

Table of Contents

Introduction

34.1 Polynomial Time

34.2 Polynomial-Time Verification

34.3 NP-Completeness

34.4 Proofs

34.5 Problems

Proving More NP-Complete Problems

We can continue to use the NP-completeness proof paradigm to prove additional problems are NP-complete. While the six steps will be there in each proof, they do not have to be as explicitly marked as in the proof of Theorem 34.10.

We now have three known NP-complete problems: SAT, CIRCUIT-SAT, and 3-CNF-SAT. We can use any of these in an NP-completeness proof. As we have more known NP-complete problems, we have more choices in future proofs.

CLIQUE

In an undirected graph $G = (V, E)$, a **clique of size** k is a subset $V' \subseteq V$ such that $|V'| = k$ and V' induces a complete subgraph of G.

The corresponding decision problem is

CLIQUE
INSTANCE: Undirected graph $G = (V, E)$; integer $k \geq 1$.
QUESTION: Does G contain a clique of size k?

The corresponding language is

$$\text{CLIQUE} = \{\rho(G, k) \mid G \text{ is an undirected graph with a clique of size } k\}.$$

Theorem 34.11

Theorem

CLIQUE *is NP-complete.*

Proof of Theorem 34.11

Follow the NP-completeness proof paradigm.

First, prove that CLIQUE $\in$ NP.

What is a certificate for CLIQUE?

Proof of Theorem 34.11

Choose 3-CNF-SAT to reduce to CLIQUE.

Let $\phi(x_1, x_2, \ldots, x_n)$ be a 3-CNF formula.

Example

$$\phi(x_1, x_2, x_3) = (x_1 \vee x_2 \vee x_3) \wedge (\neg x_1 \vee \neg x_2 \vee \neg x_3) \wedge (\neg x_1 \vee x_2 \vee \neg x_3)$$

Proof of Theorem 34.11

For simplicity, write

$$\phi(x_1, x_2, \ldots, x_n) = \\ (a_1 \vee b_1 \vee c_1) \wedge (a_2 \vee b_2 \vee c_2) \wedge \cdots \wedge (a_m \vee b_m \vee c_m).$$

Create an undirected graph $G = (V, E)$, and set $k = m$; our instance of CLIQUE will be G, k. V contains $3m$ vertices:

$$u_1, u_2, \ldots, u_m, v_1, v_2, \ldots, v_m, w_1, w_2, \ldots, w_m.$$

Label u_i with a_i, v_i with b_i, and w_i with c_i.

Proof of Theorem 34.11

Put an edge between two vertices if they do not have the same index and are not labeled with complementary literals.

Example

Here $k = 3$. There is no edge among u_1, v_1, w_1. There is no edge from u_1 to u_2, since u_1 is labeled x_1 and u_2 is labeled $\neg x_1$. There is an edge from u_1 to v_2, since u_1 is labeled x_1 and v_2 is labeled $\neg x_2$.

Draw the entire graph G on the board.

Proof of Theorem 34.11

G and k can clearly be computed in polynomial time.

We need to show that G contains a clique of size k if and only if ϕ is satisfiable.

First, assume that G has a clique of size k. Then, it is possible to have $k = m$ literals simultaneously TRUE, each in a different clause, and, hence, it is possible to make all clauses simultaneously TRUE and to make ϕ TRUE. Hence, ϕ is satisfiable.

Proof of Theorem 34.11

Second, assume that ϕ is satisfiable. Choose a satisfying truth assignment for ϕ, and, for each clause, choose a TRUE literal. Then, the corresponding k vertices are all connected by edges. Hence, G has a clique of size k.

We conclude that

$$\text{3-CNF-SAT} \leq_{\mathrm{P}} \text{CLIQUE}$$

and that CLIQUE is NP-complete.

VERTEX-COVER

In an undirected graph $G = (V, E)$, a **vertex cover of size** k is a subset $V' \subseteq V$ such that $|V'| = k$ and every edge in E has at least one of its endpoints in V'.

The corresponding decision problem is

VERTEX-COVER
INSTANCE: Undirected graph $G = (V, E)$; integer $k \geq 0$.
QUESTION: Does G contain a vertex cover of size k?

The corresponding language is

$$\text{VERTEX-COVER} = \{\rho(G, k) \mid G \text{ is an undirected graph with a vertex cover of size } k\}.$$

Theorem 34.12

Theorem

VERTEX-COVER *is NP-complete.*

Proof of Theorem 34.12

Follow the NP-completeness proof paradigm.

First, prove that VERTEX-COVER $\in$ NP.

What is a certificate for VERTEX-COVER?

Proof of Theorem 34.12

Choose CLIQUE to reduce to VERTEX-COVER.

Let G, k be an instance of CLIQUE.
Let $\overline{G} = (V, \overline{E})$ be the complement of G. The corresponding instance of VERTEX-COVER is $\overline{G}, |V| - k$.

G and k can clearly be computed in polynomial time.

Proof of Theorem 34.12

We need to show that $\overline{G}$ contains a vertex cover of size $|V| - k$ if and only if G contains a clique of size k.

First, assume that $\overline{G}$ has a vertex cover V' of size $|V| - k$. Then, $V - V'$ is a clique in G of size k. Hence, G has a clique of size k.

Proof of Theorem 34.12

Second, assume that G has a clique V' of size k. Then, $V - V'$ is a vertex cover of $\overline{G}$ of size $|V| - k$. Hence, $\overline{G}$ has a vertex cover of size $|V| - k$.

We conclude that

$$\textsc{Clique} \quad \leq_{\mathrm{P}} \quad \textsc{Vertex-Cover}$$

and that VERTEX-COVER is NP-complete.

HAMILTONIAN-CYCLE

In an undirected graph $G = (V, E)$, a **hamiltonian cycle** is a simple cycle in G that contains every vertex.

The corresponding decision problem is

HAMILTONIAN-CYCLE
INSTANCE: Undirected graph $G = (V, E)$
QUESTION: Does G contain a hamiltonian cycle?

The corresponding language is

$$\text{HAMILTONIAN-CYCLE} = \{\rho(G) \mid G \text{ is an undirected graph with a hamiltonian cycle}\}.$$

Theorem 34.13

Theorem

HAMILTONIAN-CYCLE *is NP-complete.*

Proof of Theorem 34.13

Follow the NP-completeness proof paradigm.

First, prove that HAMILTONIAN-CYCLE $\in \mathrm{NP}$.

What is a certificate for HAMILTONIAN-CYCLE?

Proof of Theorem 34.13

Choose VERTEX-COVER to reduce to HAMILTONIAN-CYCLE.

The reduction from VERTEX-COVER to HAMILTONIAN-CYCLE is a complicated construction. See the textbook.

TRAVELING-SALESMAN

In a complete, undirected graph $G = (V, E)$ with edge cost function $c : E \to \mathbb{N}$, a **traveling salesman tour** is a hamiltonian cycle $T = v_0, v_1, v_2, \ldots, v_n = v_0$ in G. The **cost** of the tour T is

$$c(T) = \sum_{i=0}^{n-1} c(v_i, v_{i+1}).$$

The corresponding decision problem is

TRAVELING-SALESMAN

INSTANCE: Complete, undirected graph $G = (V, E)$; cost function $c : E \to \mathbb{N}$; integer $k \geq 0$.

QUESTION: Does G contain a traveling salesman tour of cost $\leq k$?

TSP

The corresponding language is

$$\begin{aligned}\mathsf{TSP} \quad = \quad & \{\rho(G, c, k) \mid G \text{ is an undirected graph} \\ & \text{with a traveling salesman tour} \\ & \text{of cost } \leq k\}.\end{aligned}$$

Theorem 34.14

Theorem

TSP *is NP-complete.*

Proof of Theorem 34.14

Follow the NP-completeness proof paradigm.

First, prove that TSP $\in \mathrm{NP}$.

What is a certificate for TSP?

Proof of Theorem 34.14

Choose HAMILTONIAN-CYCLE to reduce to TSP.

Let $G = (V, E)$ be an instance of HAMILTONIAN-CYCLE.
Let $G' = (V, E')$ be the complete, undirected graph on V.
Define an edge cost function $c : E' \to \mathbb{N}$ by

$$c(u, v) = \begin{cases} 0 & \text{if } (u, v) \in E; \\ 1 & \text{if } (u, v) \notin E. \end{cases}$$

The corresponding instance of TSP is G', c, k, where $k = 0$.

G', c, and $k = 0$ can clearly be computed in polynomial time.

Proof of Theorem 34.14

Clearly, G' contains a traveling salesman tour of cost ≤ 0 if and only if G contains a hamiltonian cycle.

We conclude that

$$\textsc{Hamiltonian-Cycle} \leq_{\mathrm{P}} \text{TSP}$$

and that TSP is NP-complete.

SUBSET-SUM

Let S be a set of positive integers, and let $t > 0$ be a **target** sum. We are interested whether there is a subset $S' \subseteq S$ such that $t = \sum_{s \in S'} s$.

The corresponding decision problem is

SUBSET-SUM

INSTANCE: Set S of positive integers; target $t > 0$.

QUESTION: Does S contain a subset $S' \subseteq S$ that sums to t?

The corresponding language is

$$\text{SUBSET-SUM} = \{\rho(S, t) \mid S \text{ contains a subset summing to } t\}.$$

Example — SUBSET-SUM

Example

$$\begin{aligned} S &= \{1, 7, 8, 17, 21\} \\ t &= 23 \end{aligned}$$

This is a NO instance.

Theorem 34.15

Theorem

SUBSET-SUM *is NP-complete.*

Proof of Theorem 34.15

Follow the NP-completeness proof paradigm.

First, prove that SUBSET-SUM $\in \mathrm{NP}$.

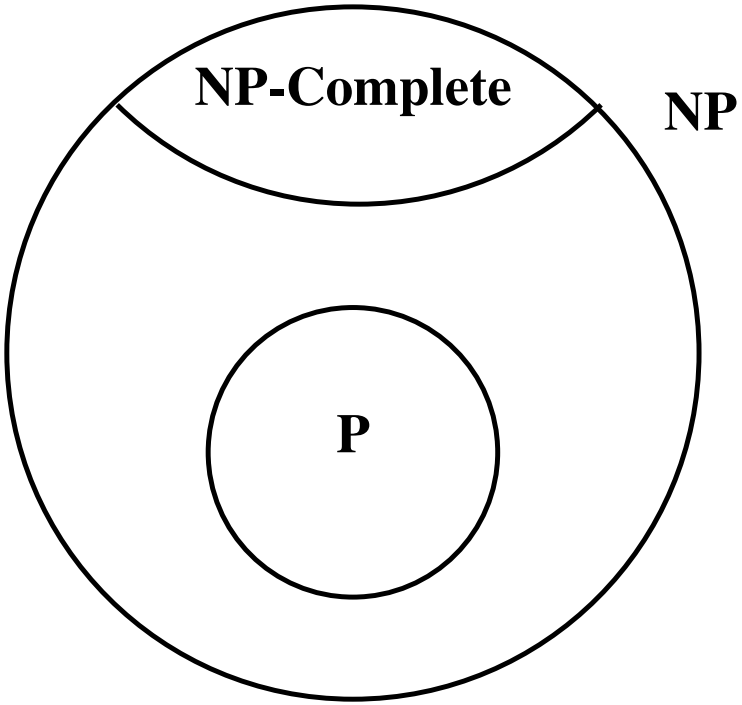
What is a certificate for SUBSET-SUM?

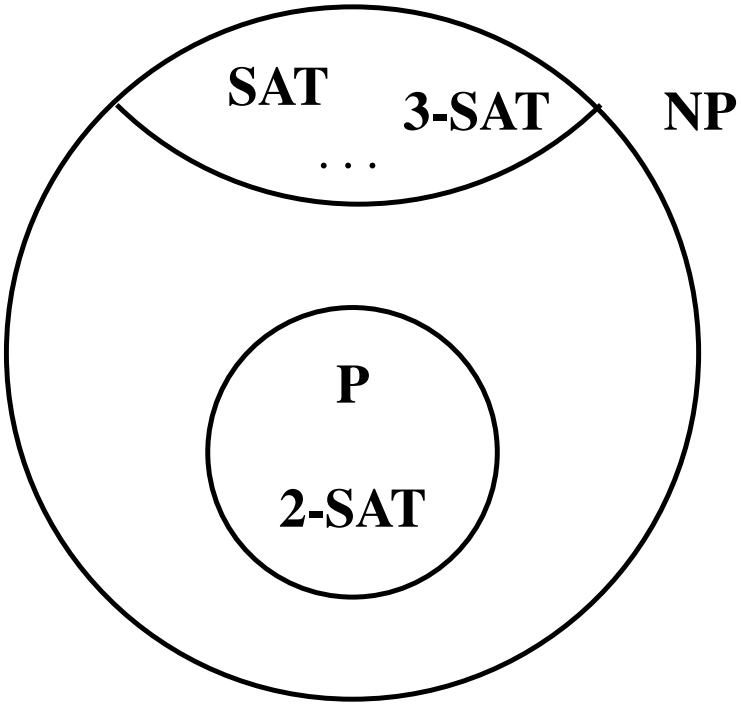
Proof of Theorem 34.15

Choose 3-CNF-SAT to reduce to SUBSET-SUM.

This reduction is particularly interesting, as it converts a logical instance to a numerical instance. Details are in the textbook.

Wrap-Up — NP-Completeness





Key concepts for defining NP-completeness are decision problems, languages, and polynomial-time reduction.